

Plan de Accion ID Industrial

Fase 1: Cimientos del Servidor (Base de Datos y API PHP)

Antes de conectar cables o diseñar interfaces, tu sistema necesita un lugar donde almacenar y procesar la información.

- **Paso 1.1: Diseño de la Base de Datos (MySQL).**
 - Crea la base de datos en tu CPanel.
 - Diseña las tablas principales: clientes, proyectos (empresas/ubicaciones) y lecturas_sensores.
 - *Nota crítica de arquitectura:* Asegúrate de que la tabla que registra los eventos de los sensores utilice explícitamente la columna `user_id` como llave foránea para vincular de manera estricta cada lectura o alerta con su cliente correspondiente.
- **Paso 1.2: API RESTful Base (PHP).**
 - Programa los endpoints esenciales en PHP:
 - `crear_cliente.php` y `asignar_sensor.php` (para el panel web).
 - `ping.php` (para recibir el "latido" periódico del ESP32 y saber que está en línea).
 - `alerta.php` (para recibir el evento de emergencia del ESP32).

Fase 2: Hardware y Firmware (ESP32-C3)

Con el servidor listo para recibir datos, es hora de darle vida al hardware.

- **Paso 2.1: Ensamblaje del Prototipo (Físico).**
 - Monta el ESP32-C3 SuperMini en la protoboard.
 - Conecta el sensor de humo (MQ-2) y el de flama a los pines analógicos/digitales.
 - Conecta el buzzer y los LEDs de estado.
- **Paso 2.2: Lógica de Detección (C++ / Arduino IDE).**
 - Escribe el código para leer los sensores de forma local y continua.
 - Programa la activación del buzzer y el LED rojo de forma autónoma (sin depender de internet) si se detecta humo o flama.
- **Paso 2.3: Conectividad y HTTP POST.**
 - Integra la librería `WiFi.h` para conectar el SuperMini a la red.
 - Usa `HTTPClient.h` para configurar el ESP32 de modo que envíe un JSON mediante POST a tu `alerta.php` **solo** cuando se active la alarma, y un POST a `ping.php` cada 5 minutos.

Fase 3: Panel Administrativo (React)

El centro de control donde se gestiona el negocio y se dan de alta los equipos.

- **Paso 3.1: Interfaces UI/UX.**
 - Desarrolla las pantallas de Login, Dashboard General, Gestión de Clientes y Estado de Sensores.
- **Paso 3.2: Consumo de la API.**
 - Conecta React con tus endpoints de PHP usando fetch o axios para permitir la creación de usuarios (user_id) y la visualización del estado del sistema.
- **Paso 3.3: Despliegue en CPanel.**
 - Ejecuta el comando de *build* en tu proyecto React y sube los archivos estáticos generados al "Administrador de Archivos" de tu CPanel.

Fase 4: Integración del Tiempo Real (Firebase)

El puente necesario para suplir la falta de WebSockets en el servidor PHP compartido.

- **Paso 4.1: Configuración de Firebase Cloud Messaging (FCM).**
 - Crea un proyecto en la consola de Firebase.
 - Obtén tus claves de servidor (Server Key / Service Account).
- **Paso 4.2: Actualización del Backend.**
 - Modifica tu archivo alerta.php en CPanel. Añade una función usando cURL para que, inmediatamente después de guardar un evento de incendio en MySQL, haga una petición a la API de Firebase y dispare una notificación Push al teléfono asociado a ese user_id.

Fase 5: Aplicación Móvil (Flutter)

La herramienta de bolsillo del cliente final.

- **Paso 5.1: Desarrollo Frontend y Autenticación.**
 - Diseña las vistas en Flutter: Login, Panel Principal (estado del edificio) y vista de Alarma Activa.
 - Conecta la app a tu API PHP para validar el inicio de sesión.
- **Paso 5.2: Implementación de Notificaciones Push.**
 - Integra el SDK de Firebase en tu proyecto Flutter.
 - Configura la app para recibir y manejar las notificaciones de FCM (Firebase Cloud Messaging) tanto en primer plano como en segundo plano. Cuando llegue una notificación de incendio, la app debe mostrar una alerta visual y sonora fuerte.

Fase 6: Automatización de Atención a Clientes

El valor agregado del sistema para ventas y soporte.

- **Paso 6.1: Configuración del NLP.**
 - Entrena un agente en Dialogflow (o la plataforma que elijas) con intenciones (intents) sobre precios, dudas técnicas y estado del servicio.
- **Paso 6.2: Webhook y Base de Datos.**
 - Crea un nuevo archivo en tu CPanel (ej. webhook_chat.php).
 - Conecta Dialogflow a este webhook. Cuando un usuario pregunte por el estado de sus equipos, el webhook consultará tu base de datos MySQL (filtrando por user_id) y le devolverá la respuesta exacta al chatbot..